SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai- 602105

Student Name: Rachitha R **Reg. No.:** 192421058

Course Code: CSA1616 **Slot:** D

Course Name: Data warehouse and Data mining

Course Faculty: Dr. Mahesh Muthulakshmi

EXP:12 1.Covariance and correlation Children of three ages are asked to indicate their preference for three photographs of adults. Do the data suggest that there is a significant relationship between age and photograph preference? What is wrong with this study? Photograph: Age of child A B C 5-6 years: 18 22 20 7-8 years: 2 28 40 9-10 years: 20 10 40 1. Use cov() to calculate the sample covariance between B and C. 2. Use another call to cov() to calculate the sample covariance matrix for the preferences. 3. Use cor() to calculate the sample correlation between B and C. 4. Use another call to cor() to calculate the sample correlation matrix for the preference.

Aim:

To calculate the sample covariance and correlation between the preferences for photographs B and C and obtain the covariance and correlation matrices for photographs A, B, and C using R programming**.**

Procedure:

1. Open R or RStudio and initialize the preference data for photographs A, B, and C using vectors.
2. Use the cov() function to calculate the sample covariance between photographs B and C.
3. Create a data frame using data.frame() to combine the three preference vectors.
4. Apply the cov() function to the data frame to obtain the sample covariance matrix.
5. Use the cor() function to calculate the sample correlation between photographs B and C.
6. Apply the cor() function to the data frame to obtain the sample correlation matrix.
7. Observe and interpret the covariance and correlation values to understand the relationship between the photograph preferences**.**

Code:

```
A <- c(18, 2, 20)
B <- c(22, 28, 10)
C <- c(20, 40, 40)
cov(B, C)
prefs <- data.frame(A, B, C)
cov(prefs)
cor(B, C)
cor(prefs)
```

Output:

```
> A <- c(18, 2, 20)
> B <- c(22, 28, 10)
> C <- c(20, 40, 40)
> cov(B, C)
[1] -20
> prefs <- data.frame(A, B, C)
> cov(prefs)
          A   B         C
A  97.33333 -74 -46.66667
B -74.00000  84 -20.00000
C -46.66667 -20 133.33333
> cor(B, C)
[1] -0.1889822
> cor(prefs)
           A          B          C
A  1.0000000 -0.8183918 -0.4096440
B -0.8183918  1.0000000 -0.1889822
C -0.4096440 -0.1889822  1.0000000
```

Result:

Thus, the sample covariance and correlation between photographs B and C, along with the covariance and correlation matrices for photographs A, B, and C, were calculated successfully using R programming.

EXP:13 .Imagine that you have selected data from the All Electronics data warehouse for analysis. The data set will be huge! The following data are a list of All Electronics prices for commonly sold items (rounded to the nearest dollar). The numbers have been sorted: 1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10, 12, 14, 14, 14, 15, 15, 15, 15, 15, 15, 18, 18, 18, 18, 18, 18, 18, 18, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 25, 25, 25, 25, 25, 28, 28, 30, 30, 30. (i) Partition the dataset using an equal-frequency partitioning method with bin equal to 3 (ii) Apply data smoothing using bin means and bin boundary. (iii) Plot Histogram for the above frequency division

Aim:

To partition the given price dataset into three approximately equal-frequency bins, perform data smoothing using bin means and bin boundaries, and plot a histogram using R programming.

Procedure:

1. Open R or RStudio and enter the given sorted price data into a vector using the c() function.
2. Calculate the total number of observations using the length() function.
3. Divide the dataset into three approximately equal-frequency bins using the split() function and appropriate sequence indexing.
4. Apply data smoothing using bin means by calculating the mean of each bin with the mean() function and replacing the values in each bin with its mean.
5. Apply data smoothing using bin boundaries by identifying the minimum and maximum values of each bin and replacing each observation with its nearest boundary.
6. Use the hist() function to plot a histogram of the original price data.
7. Add a suitable title and label the X-axis as Price and the Y-axis as Frequency.
8. Execute the program and observe the bin partitions, smoothed values, and histogram

Code:

```r
price <- c(1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10,
           12, 14, 14, 14, 15, 15, 15, 15, 15, 15,
           18, 18, 18, 18, 18, 18, 18, 18, 20, 20,
           20, 20, 20, 20, 21, 21, 21, 21,
           25, 25, 25, 25, 25, 28, 28, 30, 30, 30)
n <- length(price)
bin_size <- n / 3
bins <- split(price, ceiling(seq_along(price) / bin_size))
bins
bin_means <- lapply(bins, function(b) {
  rep(mean(b), length(b))
})
bin_means
bin_boundary <- lapply(bins, function(b) {
  lower <- min(b)
  upper <- max(b)

  sapply(b, function(x) {
    if (abs(x - lower) < abs(x - upper))
      lower
    else
      upper
  })
})
bin_boundary
hist(price,
     main = "Price Histogram",
     xlab = "Price",
     ylab = "Frequency")
```

Output:

```
> price <- c(1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10,
+            12, 14, 14, 14, 15, 15, 15, 15, 15, 15,
+            18, 18, 18, 18, 18, 18, 18, 18, 20, 20,
+            20, 20, 20, 20, 21, 21, 21, 21,
+            25, 25, 25, 25, 25, 28, 28, 30, 30, 30)
> n <- length(price)
> bin_size <- n / 3
> bins <- split(price, ceiling(seq_along(price) / bin_size))
> bins
$`1`
 [1]  1  1  5  5  5  5  5  8  8 10 10 10 10 12 14 14 14

$`2`
 [1] 15 15 15 15 15 15 18 18 18 18 18 18 18 18 20 20 20

$`3`
 [1] 20 20 20 20 21 21 21 21 25 25 25 25 25 28 28 30 30 30

> bin_means <- lapply(bins, function(b) {
+   rep(mean(b), length(b))
+ })
> bin_means
$`1`
 [1] 8.058824 8.058824 8.058824 8.058824 8.058824 8.058824
 [7] 8.058824 8.058824 8.058824 8.058824 8.058824 8.058824
[13] 8.058824 8.058824 8.058824 8.058824 8.058824

$`2`
 [1] 17.29412 17.29412 17.29412 17.29412 17.29412 17.29412
 [7] 17.29412 17.29412 17.29412 17.29412 17.29412 17.29412
[13] 17.29412 17.29412 17.29412 17.29412 17.29412

$`3`
 [1] 24.16667 24.16667 24.16667 24.16667 24.16667 24.16667
 [7] 24.16667 24.16667 24.16667 24.16667 24.16667 24.16667
[13] 24.16667 24.16667 24.16667 24.16667 24.16667 24.16667

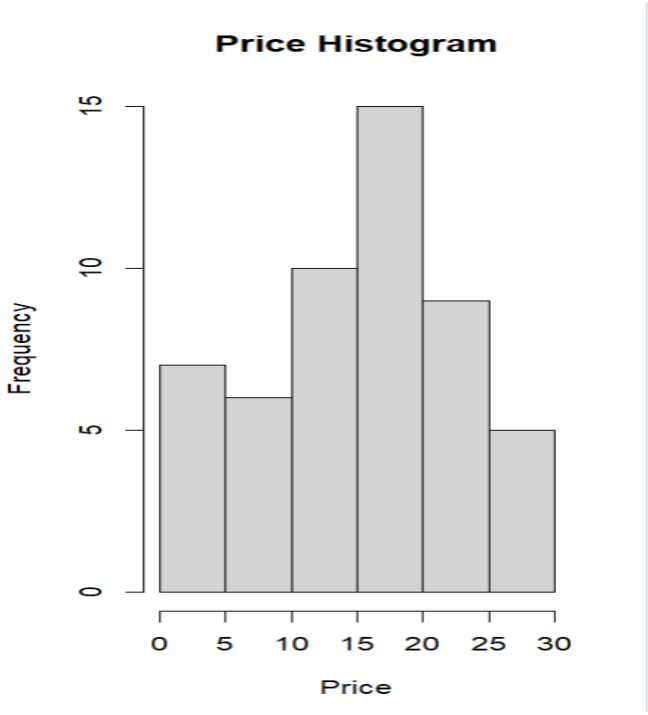
> bin_boundary <- lapply(bins, function(b) {
```

```
> bin_boundary <- lapply(bins, function(b) {
+   lower <- min(b)
+   upper <- max(b)
+
+   sapply(b, function(x) {
+     if (abs(x - lower) < abs(x - upper))
+       lower
+     else
+       upper
+   })
+ })
> bin_boundary
$`1`
 [1]  1  1  1  1  1  1  1 14 14 14 14 14 14 14 14 14 14

$`2`
 [1] 15 15 15 15 15 15 20 20 20 20 20 20 20 20 20 20 20

$`3`
 [1] 20 20 20 20 20 20 20 20 30 30 30 30 30 30 30 30 30 30

> hist(price,
+      main = "Price Histogram",
+      xlab = "Price",
+      ylab = "Frequency")
>
```



Result:

Thus, the given price dataset was partitioned into three approximately equal-frequency bins, data smoothing was performed using bin means and bin boundaries, and the histogram was plotted successfully using R programming.